

Programação Orientada a Objetos

Classes, objetos, atributos e métodos.

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



- 1 Relembrando o paradigma Orientação a Objetos
- 2 Classes
- 3 Atributos de uma classe
- 4 Métodos
- 5 Método Construtor
- 6 Posicionamento no código JAVA
- 7 Código completo com explicação

Paradigma de Orientação a Objetos

Definição

O paradigma de **Orientação a Objetos (OO)** é um modelo de programação baseado na organização do software em **objetos**, que representam entidades do mundo real ou conceitos abstratos.

Cada objeto é composto por:

- **Estado** (atributos)
- **Comportamento** (métodos)

Conceitos Fundamentais

- **Classe**: molde ou modelo que define atributos e métodos.
- **Objeto**: instância concreta de uma classe.
- **Encapsulamento**: proteção e controle de acesso aos dados.

Objetivo do Paradigma

Promover:

- Modularidade
- Reutilização de código

Paradigma Orientado a Objetos

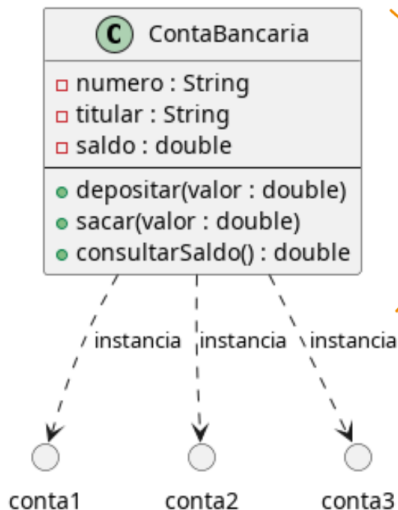
O paradigma orientado a objetos (POO) organiza o software em **classes**, que são moldes para os **objetos**, entidades do mundo real.



Figura: Classe



Figura: Objeto



Classe

Sua definição
ocorre em tempo
de compilação

Objetos

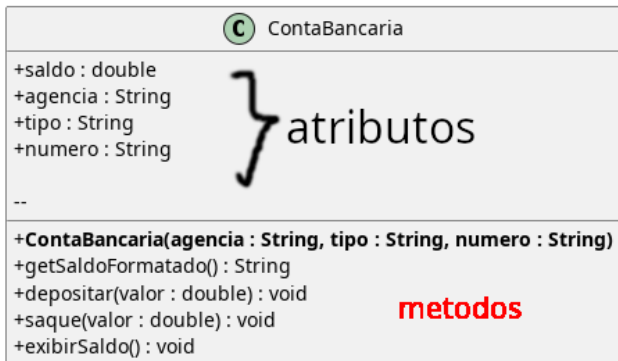
Sua criação
ocorre em tempo
de execução

Definição Formal

Uma **classe** é uma estrutura abstrata que define um tipo de objeto. Ela especifica:

- O conjunto de **atributos** (estado)
- O conjunto de **métodos** (comportamento)

A classe funciona como um *molde* para a criação de objetos.



Classe em Orientação a Objetos

Main.java

ContaBancaria.java x

```
1 public class ContaBancaria { no usages
2
3     // Atributos (conforme UML)
4     public double saldo; no usages
5     public String agencia; no usages
6     public String tipo;
7     public String numero; no usages
8
9     public ContaBancaria(String agencia, String tipo, String numero) { no usages
10
11     }
12 >     public String getSaldoFormatado() {}
15 >     public void depositar(double valor) {}
18 >     public void saque(double valor) {}
21 >     public void exibirSaldo() {}
24 }
25
```



Atributos na Classe ContaBancaria

Definição no Contexto da Classe

Atributos são variáveis declaradas dentro de uma classe que vão representar o **estado** dos objetos criados a partir dela. Na classe ContaBancaria, os atributos representam as informações que caracterizam cada conta criada no sistema.

Função dos Atributos

Eles armazenam o **estado da conta**, ou seja:

- Quanto dinheiro ela possui
- A qual agência pertence
- Qual o seu tipo
- Qual o seu número identificador

Importante

Cada objeto criado a partir da classe ContaBancaria possui seus próprios valores para esses atributos, garantindo independência entre as contas.



Declaração na Classe

- `public double saldo;`
- `public String agencia;`
- `public String tipo;`
- `public String numero;`

Análise Técnica

- **saldo** *double*: armazena valores numéricos com casas decimais.
- **agencia** *String*: identifica a unidade bancária.
- **tipo** *String*: classifica a conta (corrente, poupança).
- **numero** *String*: identifica unicamente a conta.

Interpretação

Esses atributos definem o **estado interno** de cada instância da classe.





atributos

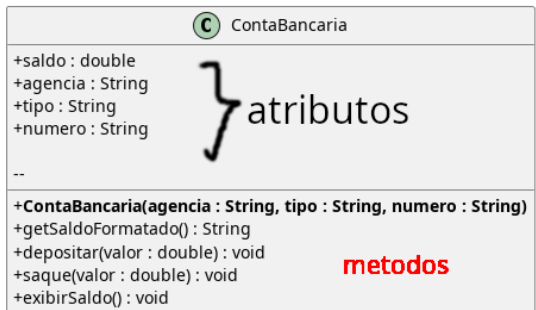


Figura: Representação ilustrativa de uma classe em UML

Definição

Métodos são funções definidas dentro de uma classe que representam o **comportamento** dos objetos.

Métodos em Orientação a Objetos

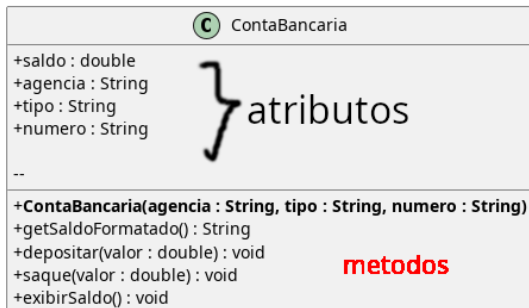
Função dos Métodos

- Manipular os atributos (estado)
- Executar operações relacionadas ao objeto
- Definir o que o objeto é capaz de fazer

Estrutura Geral

modificador retorno nomeMetodo(parametros)

Exemplo: `public void depositar(double valor)`



Construtor

```
ContaBancaria(String agencia, String tipo, String numero)
```

Métodos Declarados

- `getSaldoFormatado() : String`
- `depositar(double valor) : void`
- `saque(double valor) : void`
- `exibirSaldo() : void`

Observação

Esses métodos definem as operações que podem ser realizadas sobre uma conta bancária.

getSaldoFormatado()

Retorna o valor do saldo convertido para uma representação textual formatada (moeda).

depositar(double valor)

Adiciona um valor ao atributo `saldo`, alterando o estado interno da conta.

saque(double valor)

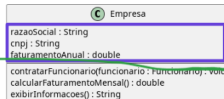
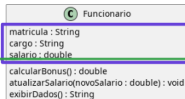
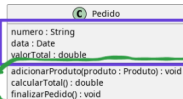
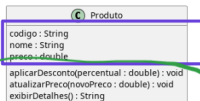
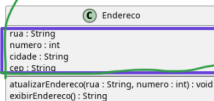
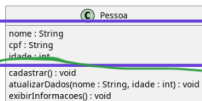
Subtrai um valor do `saldo`, caso haja saldo suficiente.

exibirSaldo()

Apresenta ao usuário o saldo atual da conta.

métodos

atributos



Método Construtor

Definição

O **método construtor** é um método especial de uma classe responsável por **inicializar os atributos do objeto no momento de sua criação**.

Características

- Possui o **mesmo nome da classe**.
- É executado automaticamente quando o objeto é instanciado.
- Normalmente define o estado inicial dos atributos.
- Pode receber parâmetros.

Exemplo Geral em Java

```
public class Pessoa {  
    private String nome;  
  
    public Pessoa(String nome) {  
        this.nome = nome;  
    }  
}
```



```
public class ContaBancaria {  
  
    private String titular;  
    private int numero;  
    private double saldo;  
  
    public ContaBancaria(String titular, int numero) {  
        this.titular = titular;  
        this.numero = numero;  
        this.saldo = 0.0;  
    }  
}
```

Identificação

- O método ContaBancaria(...) é o construtor
- Nome idêntico ao da classe
- Inicializa os atributos



Posicionamento no código JAVA

```
public class ContaBancaria { no usages
```

```
// Atributos (conforme UML)
```

```
public double saldo; no usages
```

```
public String agencia; no usages
```

```
public String tipo;
```

```
public String numero; no usages
```

```
// Construtor
```

```
public ContaBancaria(String agencia, String tipo, String numero) { no usages
```

```
    this.agencia = agencia;
```

```
    this.tipo = tipo;
```

```
    this.numero = numero;
```

```
    this.saldo = 0.0;
```

```
}
```

```
// Retorna saldo formatado em moeda brasileira
```

```
public String getSaldoFormatado() { no usages
```

```
    // implementação do método getSaldoFormatado
```

```
}
```

```
// Método para depósito
```

```
public void depositar(double valor) { no usages
```

```
    // implementação do método depositar
```

```
}
```

```
// Método para saque
```

```
public void saque(double valor) { no usages
```

```
    // implementação do método saque
```

Nome da classe

Atributos

Método
construtor

Implementação
dos demais
métodos

Código completo com explicação: parte 1

```
1  import java.text.NumberFormat;
2  import java.util.Locale;
3
4  public class ContaBancaria { no usages
5
6      // ===== Atributos Públicos =====
7      public double saldo; no usages
8      public String agencia; no usages
9      public String tipo;
10     public String numero; no usages
11
12     // ===== Construtor =====
13     public ContaBancaria(String agencia, String tipo, String numero) { no usages
14         this.agencia = agencia;
15         this.tipo = tipo;
16         this.numero = numero;
17         this.saldo = 0.0;
18     }
19 }
```

Definição

No início do código há a importação de códigos de terceiros com `import java.text.NumberFormat;` e `import java.util.Locale;`, que são bibliotecas para formatação de números e localização. Depois vem a declaração da classe `ContaBancaria` e a definição de seus atributos, seguido do método construtor.

Código completo com explicação: parte 1

```
Main.java  ContaBancaria.java x
1  import java.text.NumberFormat;
2  import java.util.Locale;
3
4  public class ContaBancaria { no usages
5
6      // ===== Atributos Públicos =====
7      public double saldo; no usages
8      public String agencia; no usages
9      public String tipo;
10     public String numero; no usages
11
12     // ===== Construtor =====
13     public ContaBancaria(String agencia, String tipo, String numero) { no usages
14         this.agencia = agencia;
15         this.tipo = tipo;
16         this.numero = numero;
17         this.saldo = 0.0;
18     }
19
```

Definição

Observe que o método construtor tem o mesmo nome da classe e é responsável por inicializar os atributos `agencia`, `tipo` e `numero` com os valores passados como parâmetros, além de definir o `saldo` inicial como zero. Isso é interessante a fim de garantir que toda conta criada tenha um estado inicial consistente.

Código completo com explicação: parte 2

```
// ===== Construtor =====
public ContaBancaria(String agencia, String tipo, String numero) { no usages
    this.agencia = agencia;
    this.tipo = tipo;
    this.numero = numero;
    this.saldo = 0.0;
}

// ===== Métodos =====

public void depositar(double valor) { no usages
    if (valor > 0) {
        saldo = saldo + valor;
        System.out.println("Depósito realizado com sucesso.");
    } else {
        System.out.println("Valor inválido para depósito.");
    }
}

public void sacar(double valor) { no usages
    if (valor > 0 && valor <= saldo) {
        saldo = saldo - valor;
        System.out.println("Saque realizado com sucesso.");
    } else {
        System.out.println("Saldo insuficiente ou valor inválido.");
    }
}
```

Definição

Logo abaixo há a implementação dos demais métodos da classe, como `getSaldoFormatado()`, que utiliza a biblioteca `NumberFormat` para formatar o saldo como moeda, e os métodos `depositar`, `sacar` e `exibirSaldo`, que manipulam o estado da conta e apresentam informações ao usuário.

Código completo com explicação: parte 3

```
29     }
30
31     public void sacar(double valor) { no usages
32         if (valor > 0 && valor <= saldo) {
33             saldo = saldo - valor;
34             System.out.println("Saque realizado com sucesso.");
35         } else {
36             System.out.println("Saldo insuficiente ou valor inválido.");
37         }
38     }
39
40     public void exibirSaldo() { no usages
41         NumberFormat formato = NumberFormat.getCurrencyInstance(new Locale("pt", "BR"));
42         System.out.println("Saldo atual: " + formato.format(saldo));
43     }
44
45     public void exibirDadosConta() { no usages
46         System.out.println("Agência: " + agencia);
47         System.out.println("Tipo: " + tipo);
48         System.out.println("Número: " + numero);
49         exibirSaldo();
50     }
51     > System.out.println("-----");
52 }
53
```

Definição

Logo abaixo há a implementação dos demais métodos da classe, como `getSaldoFormatado()`, que utiliza a biblioteca `NumberFormat` para formatar o saldo como moeda, e os métodos `depositar`, `saque` e `exibirSaldo`, que manipulam o estado da conta e apresentam informações ao usuário.

Código completo com explicação: parte 4 (FINAL)

```
public class Main {  
  
    public static void main(String[] args) {  
  
        ContaBancaria conta1 = new ContaBancaria("001", "Corrente", "12345-6");  
        ContaBancaria conta2 = new ContaBancaria("002", "Poupança", "98765-4");  
  
        conta1.depositar(1500);  
        conta1.sacar(300);  
        conta1.transferir(500, conta2);  
  
        conta1.exibirDadosConta();  
        conta2.exibirDadosConta();  
  
        // Acesso direto aos atributos (como você pediu)  
        conta1.saldo = 9999; // Alteração direta do saldo  
        System.out.println("Saldo alterado diretamente: " + conta1.saldo);  
    }  
}
```

Definição

Aqui temos a criação de duas instâncias da classe `ContaBancaria`, representando duas contas bancárias distintas. Cada conta tem seus próprios valores para os atributos, demonstrando a independência entre os objetos criados a partir da mesma classe.

